

基于 MySQL 的 Spring 应用的读写效率

陈浩, 宋松原, 吴斐, 肖文哲, 霍彦儒, 刘陈清

摘要: 在基于数据库的应用中,数据库访问是系统的主要瓶颈之一.本次实验旨在通过 JMeter 对基于 Spring 应用中 MyBatis 对数据库的读写效率进行对比测试.实验中针对两个 RESTful API 进行测试,并通过分析响应时间百分比分布和响应时间趋势来衡量系统的性能.实验结果表明,读操作的效率优于写操作,特别是在高并发场景下,写操作由于数据持久化、锁定机制等因素,响应时间显著增加.

关键词: MySQL、Spring 应用、MyBatis、JMeter

Read and write efficiency of Spring applications based on MySQL

CHEN Hao, SONG Song-Yuan, WU Fei, XIAO Wen-Zhe, HUO Yan-Ru, LIU Chen-Qing

Abstract: In database-based applications, database access is one of the main bottlenecks of the system. This experiment aims to use JMeter to comparatively test the read and write efficiency of MyBatis on the database in Spring-based applications. In the experiment, two RESTful APIs were tested, and the performance of the system was measured by analyzing the response time percentage distribution and response time trend. Experimental results show that the efficiency of read operations is better than that of write operations. Especially in high-concurrency scenarios, the response time of write operations increases significantly due to factors such as data persistence and locking mechanisms.

Key words: MySQL; Spring Application; MyBatis; JMeter

1 问题描述

在数据库驱动的应用程序中,数据库的读写效率是系统性能的关键影响因素之一.特别是在高并发场景下,数据库的性能瓶颈常常成为整个系统的瓶颈.为了解 MySQL 数据库在 Spring 应用中的读写效率表现,尤其是在高并发负载下的表现,本实验设计了多个场景,通过 JMeter 对 MyBatis 数据库访问层进行性能测试.实验的核心目标是对比系统在处理大量并发读请求与写请求时的性能差异,分析其响应时间,并找到系统能够有效处理请求的临界点,从而为应用的性能优化提供参考依据.

2 实验分析

2.1 实验数据

实验环境由四台虚拟机组成,分别为:

- 服务器 A: Ubuntu 18.04,2 核 1G 内存,作为管理机,用于编译 productdemoaop.
- 服务器 B: Ubuntu 18.04,2 核 1G 内存,部署 productdemoaop 应用.
- 服务器 C: Ubuntu 18.04,2 核 1G 内存,部署 MySQL 数据库.
- 服务器 D: Ubuntu 18.04,2 核 1G 内存,安装 JMeter 5.6.3,用于测试.

2.2 实验方法

本实验首先在本地机用 Jmeter 的 GUI 界面定义好 xxx.jmx 的测试配置,然后在服务器 D 上以非 GUI 模式下运行测试,并将结果保存在 xxx.jtl 文件中,然后利用 xxx.jtl 生成可视化的 HTML 报告.

测试内容包括对两个 RESTful API 的读写操作.通过定义不同的并发线程数和测试循环数和必要的运行时参数,记录响应时间.测试关注的主要指标是响应时间趋势图 (Response Times Over Time) 和响应时间百分比分布图 (Response Time Percentiles),并根据这些数据分析系统性能.

Response Times Over Time 图表显示了整个测试期间各个请求的响应时间变化趋势,使得能够可视化地了解在不同时间点或测试不同阶段,系统响应请求的速度如何变化。

Response Time Percentiles 响应时间百分比分布曲线图即响应时间在某个范围内的请求在所有请求数中所占的比率,相比于平均响应时间,这个值更适合用来衡量系统的稳定性。

2.3 实验设计

在本次实验中,我们设定响应时间的目标值为 40ms 左右,具体依据如下:

首先,人的感知速度对于响应时间的敏感度是一个重要因素。研究表明,人类大脑处理视觉输入的速度大约在 13 到 80 毫秒之间,因此在这一时间范围内的任何响应,对于大多数用户来说,都是即时的。40 毫秒的响应时间基本上可以被用户感知为非常迅速且流畅。同时,在 30FPS 的情况下,40 毫秒的响应时间大致与一帧的时间相当,在此范围内,系统的延迟不会被用户察觉。

其次,我们要求应用系统在 80% 的请求中,响应时间能够维持在 40 毫秒以内。这表明系统不仅能够高效处理请求,还具备充足的资源来应对当前规模的负载。

为了确保结果的可靠性,本次实验中的每个测试将多次运行,以避免单次测试带来的偶然性。

JMeter 测试参数与性能调整

在 JMeter 测试^[1]中,关键的配置参数包括线程数 (thread) 和循环数 (loop)。线程数决定了并发用户的数量,而循环数则定义了每个线程重复执行测试的次数。较高的线程数和循环数往往会对系统带来较大的负载压力,因此在高并发测试中,这两个参数需要合理设置,以便获得有效的性能测试结果。

由于实验所使用的测试云服务器 D 仅配备了 2 核 CPU 和 1GB 内存,我们遇到了性能瓶颈。内存的限制使得 JMeter 默认配置下无法有效运行。为了应对这一问题,我们修改了 JMeter 的 JVM 配置,降低堆空间以节省内存。具体修改如下:

```
export JVM_ARGS="-Xms256m -Xmx256m -XX:MaxMetaspaceSize=256m"
```

这一配置将 JVM 堆内存最小和最大值均设置为 256MB,同时限制了元空间的大小,以确保在内存受限的情况下 JMeter 能够正常启动。

此外,为了进一步提高能够开启的线程数量,我们参考了一些文献并了解到,通过降低 stacksize 可以有效增加线程的数量^[2]。这是因为每个线程在启动时都会分配一定的栈空间,而减少栈空间可以减小每个线程的内存占用,从而提高总线程数。经过实验,降低栈空间确实提高了系统能够开启的最大线程数,为我们进行更大规模的并发测试提供了可能性。

尽管通过降低 stacksize 可以增加 JMeter 的线程数,但线程数依然存在上限。为此,我们选择通过增加 loop 次数来提高并发请求数。在应用服务器上,由于请求总数和每秒的请求数保持不变,因此这种调整对于应用服务器来说是不可见的,应用只会处理相同数量的请求,而不关心线程和 loop 的配置差异。

2.4 实验结果与分析

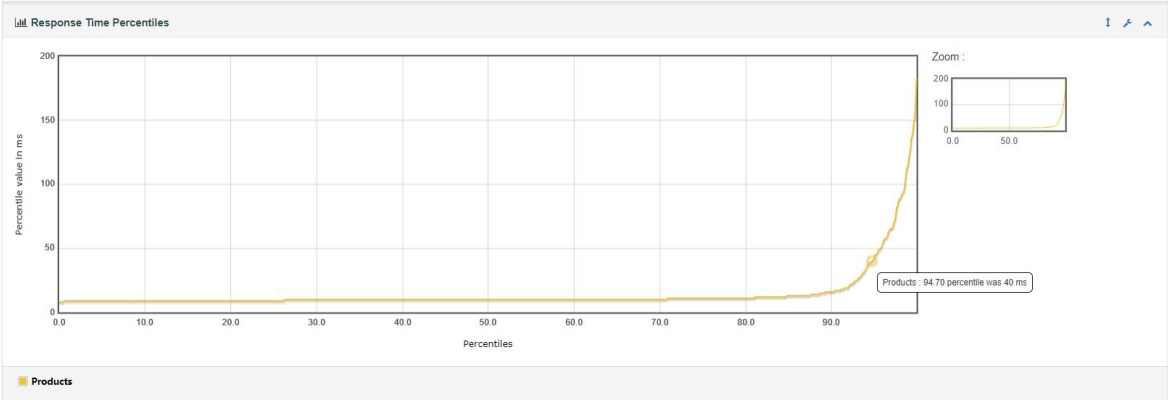
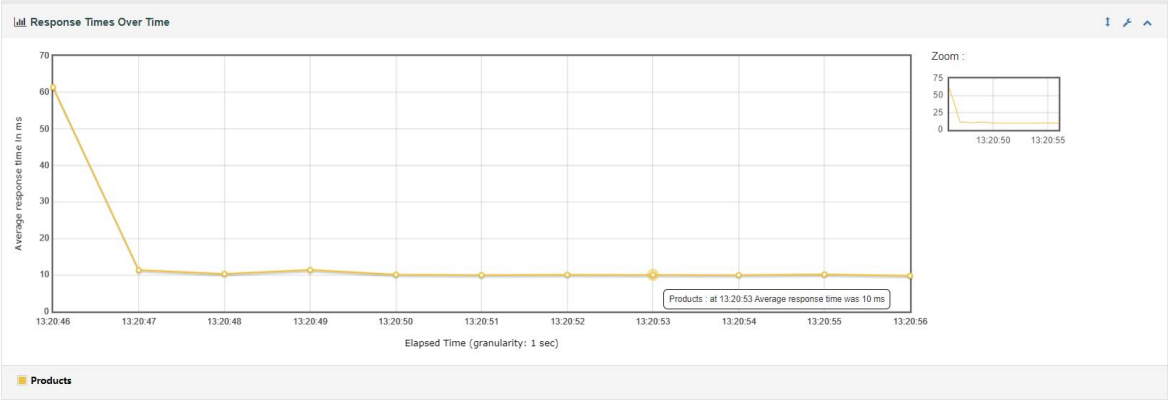
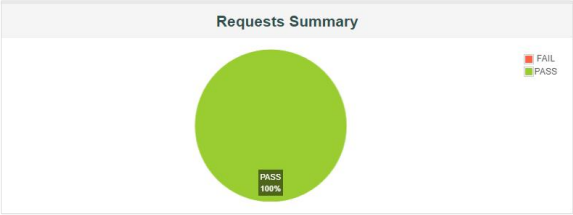
2.4.1 ReadProduct API 测试

在本次测试中,我们针对读操作执行了多个请求负载的对比测试,分别从 2000 请求到 2400 请求进行测试。每个测试组的配置 (x-y-z) 分别代表着线程数、ramp-up 时间和 loop 数,并且所有的 Response Times Over Time 均去除了 TCP/IP 连接带来的影响。

1000-10-2:

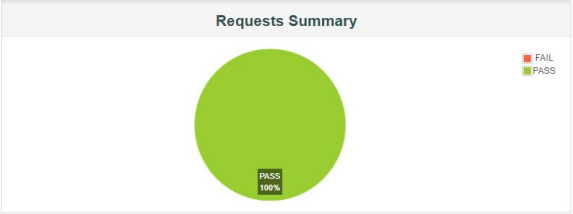
- Response Times Over Time 非常稳定,保持在 10-20ms 范围内,表明系统在此并发负载下能够轻松处理请求。
- Response Time Percentiles 显示 95% 的请求响应时间都在 40ms 内,性能表现良好。

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	Products

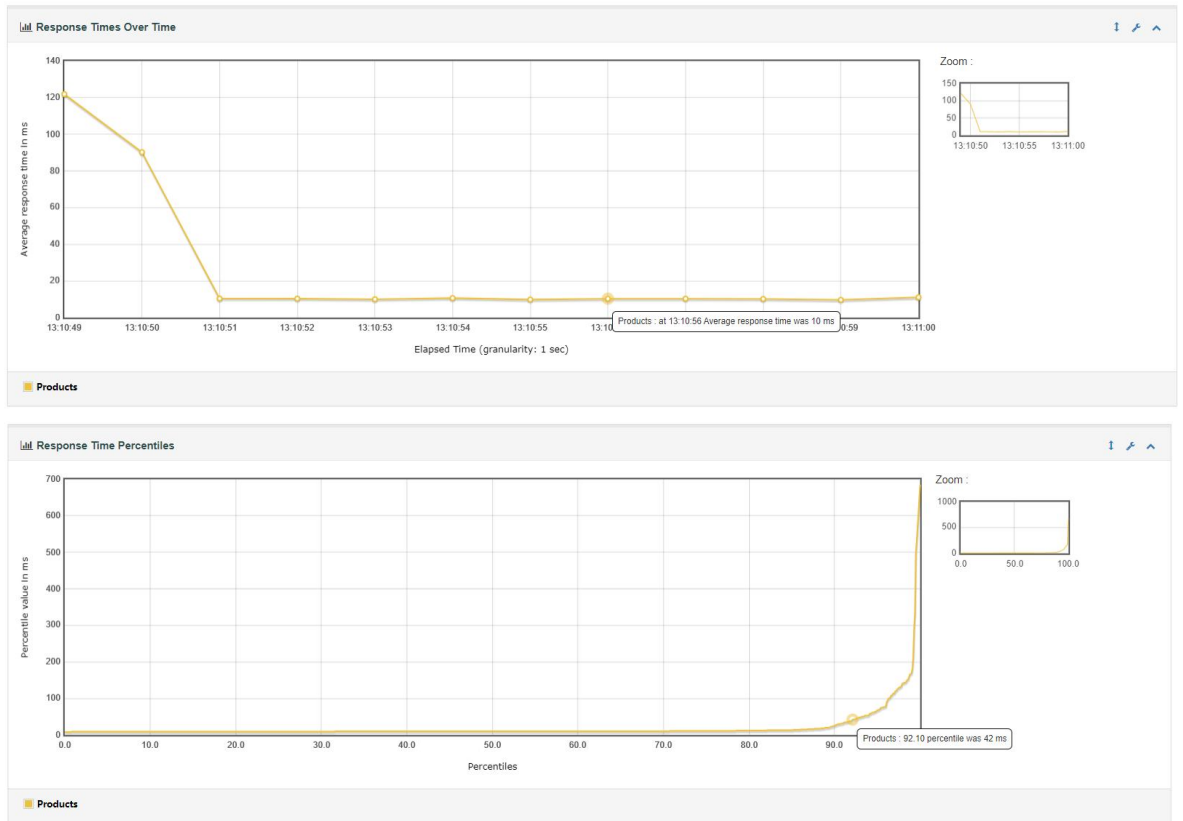


- 1050-10-2:
- Response Times Over Time 依然稳定,保持在 10-20ms.
 - Response Time Percentiles 表现略有下降,但 92% 的请求仍在 40ms 内,系统仍能很好地应对此负载.

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.998	500 ms	1 sec 500 ms	Total
0.998	500 ms	1 sec 500 ms	Products

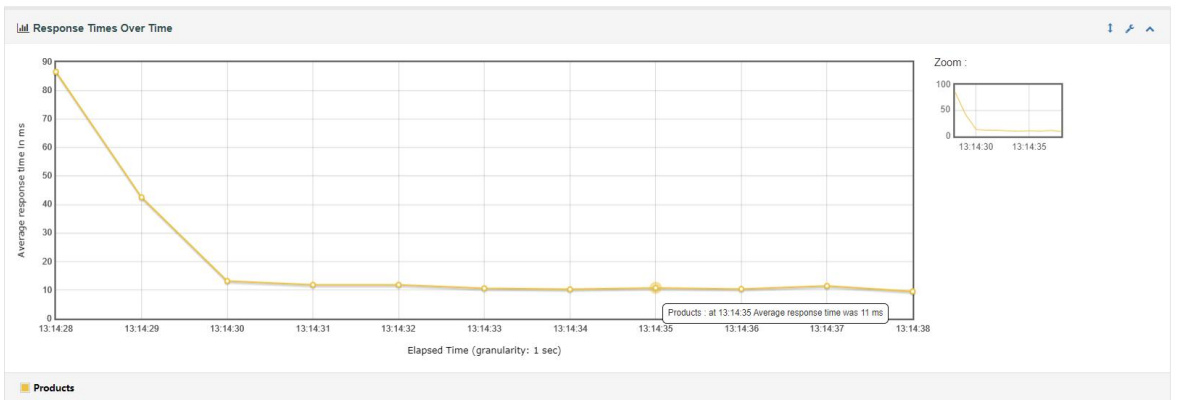
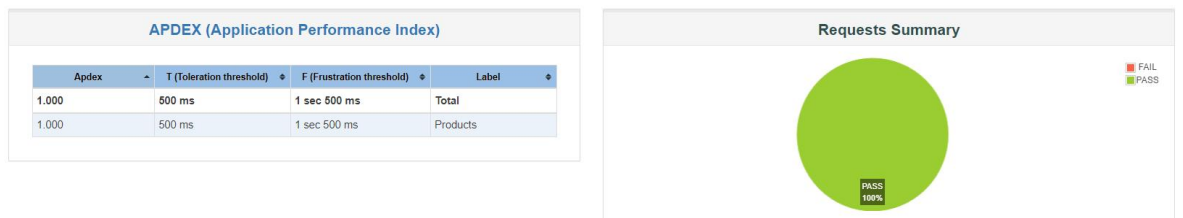


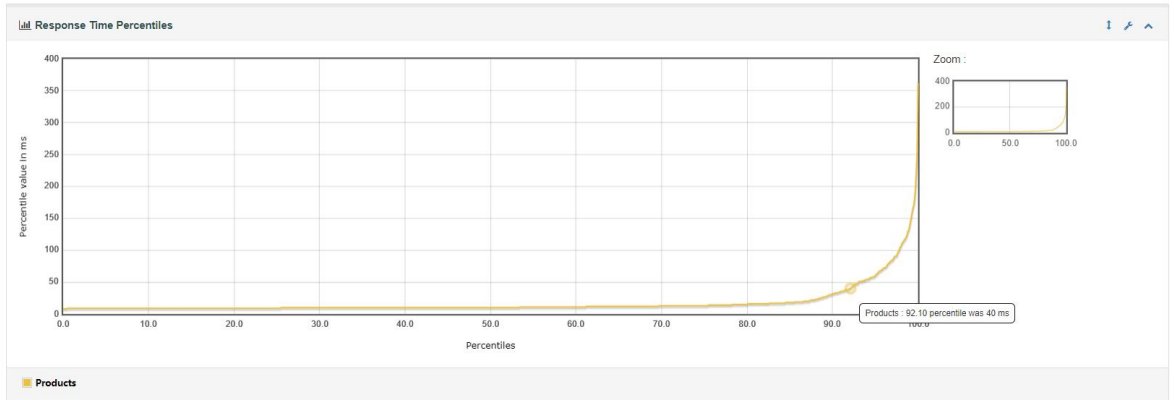
基于 MySQL 的 Spring 应用的读写效率



1100-10-2:

- Response Times Over Time 保持稳定在 10-20ms.
- Response Time Percentiles 依然有 92% 的请求在 40ms 内,系统表现与 2100 请求时相当.



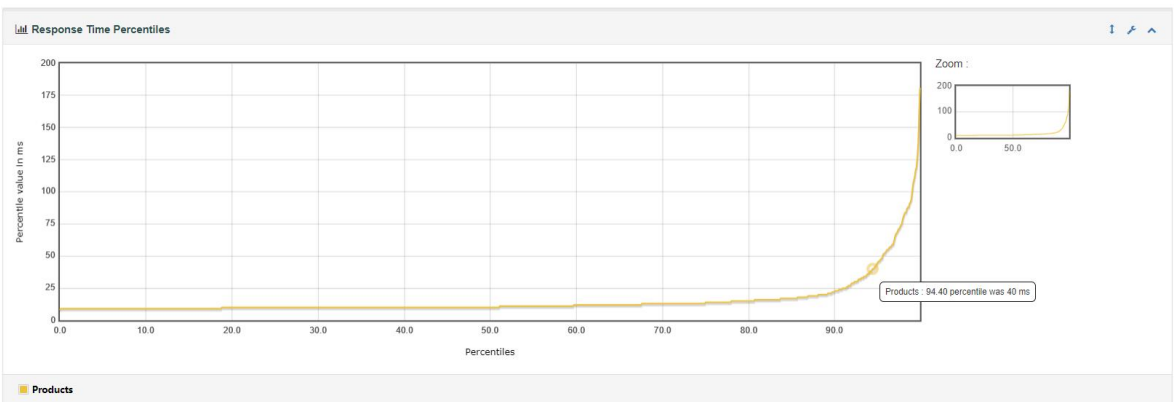
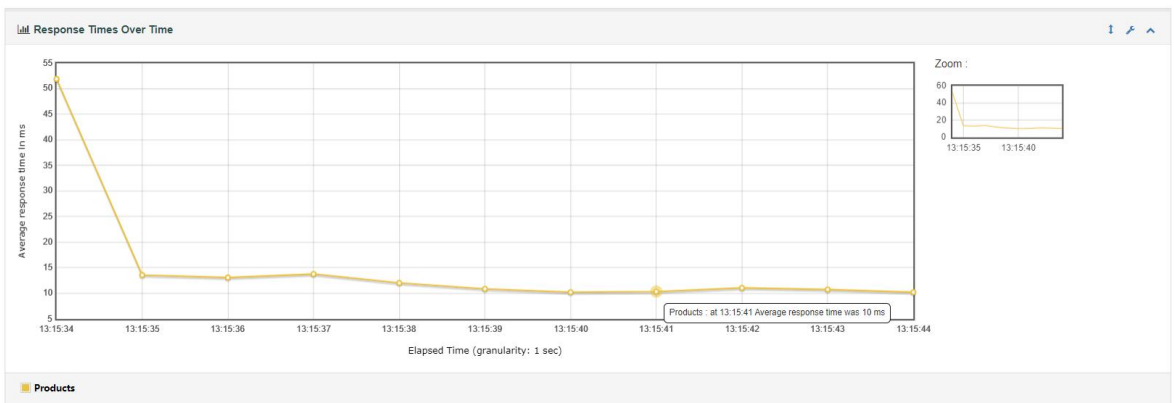
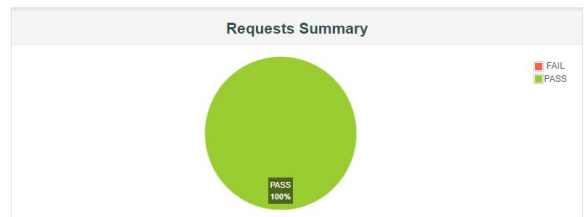


1150-10-2:

- Response Times Over Time 继续保持稳定在 10-20ms,表明系统在此负载下的表现仍然良好.
- Response Time Percentiles 提升至 94%,仍保持大部分请求响应在 40ms 内.

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	Products



1200-10-2:

- Response Times Over Time 发生明显变化,出现了几百甚至上千毫秒的延迟,系统开始出现明显的性能瓶颈.
- Response Time Percentiles 显示 50% 的请求响应时间在 600ms 左右,系统在此负载下明显无法维持之前的高效处理,响应时间急剧上升.



因此,可以推断出系统在 1150-10-2 读请求的并发负载下,是其能有质量地处理请求的最大临界点.在此并发负载下,系统仍然能够维持良好的响应时间和稳定性,超过此负载,系统的响应时间和性能将显著下降.

2.4.2 WriteProduct API 测试

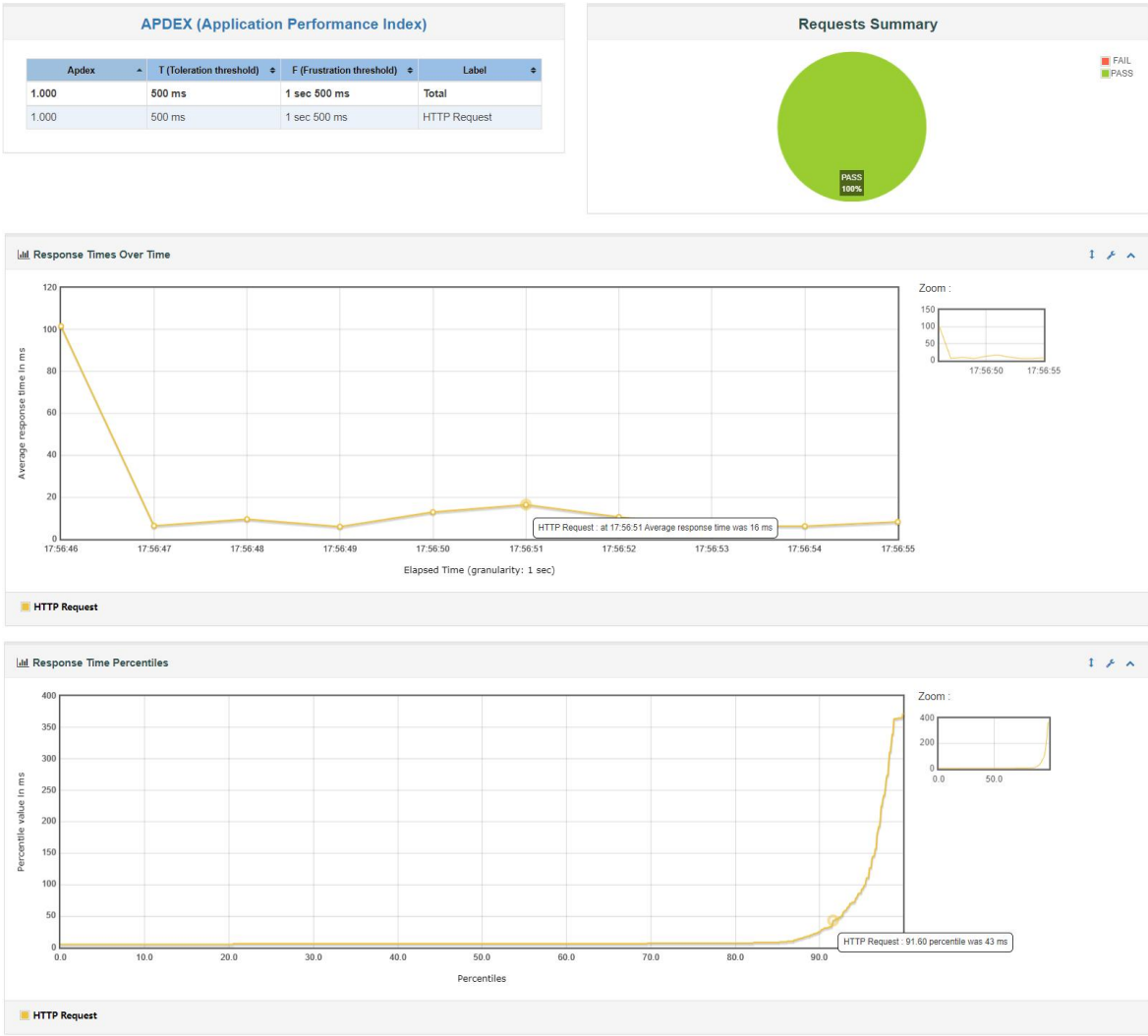
在本次测试中,我们针对写操作执行了多个请求负载的对比测试,分别从 1000 请求到 1300 请求进行测试.每个测试组的配置 (x-y-z) 分别代表着线程数、ramp-up 时间和 loop 数,并且所有的 Response Times Over Time 均去除了 TCP/IP 连接带来的影响.

1000-10-1

- Response Times Over Time 非常稳定,保持在 10-20ms 范围内,表明系统在此并发负载下能够轻松处

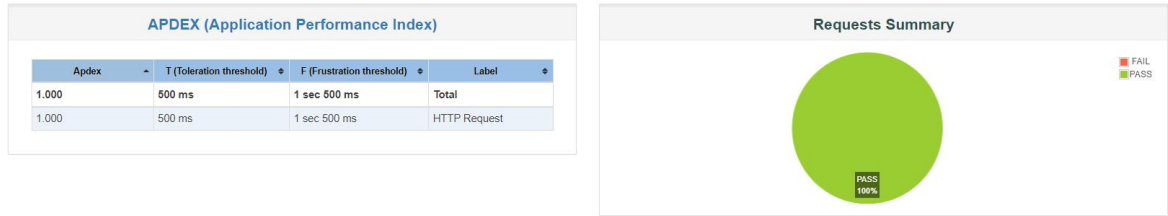
理写请求.

- Response Time Percentiles 显示 91% 的请求响应时间都在 40ms 内,系统表现优异.

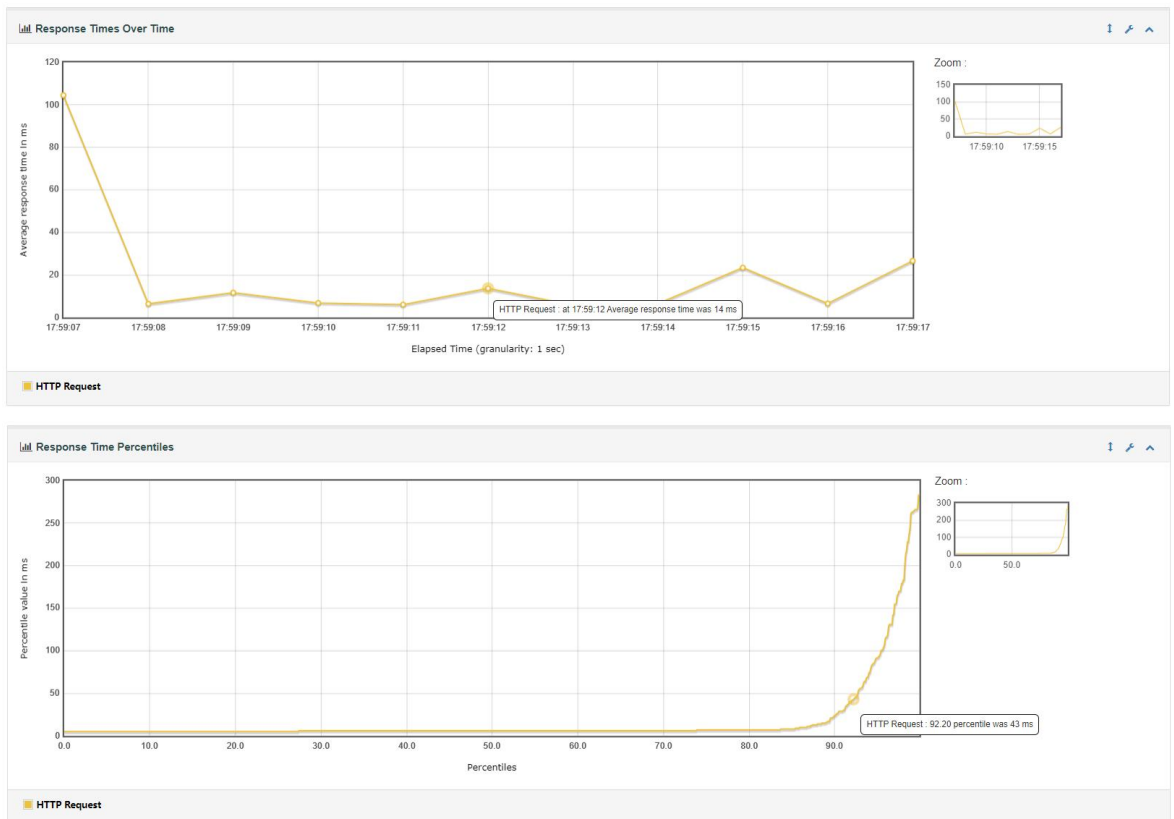


1100-10-1

- Response Times Over Time 依然稳定,保持在 10-20ms.
- Response Time Percentiles 显示 92% 的请求响应时间在 40ms 内,系统应对此负载的表现良好,稍有提升.



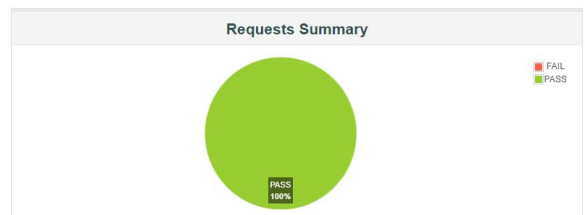
基于 MySQL 的 Spring 应用的读写效率

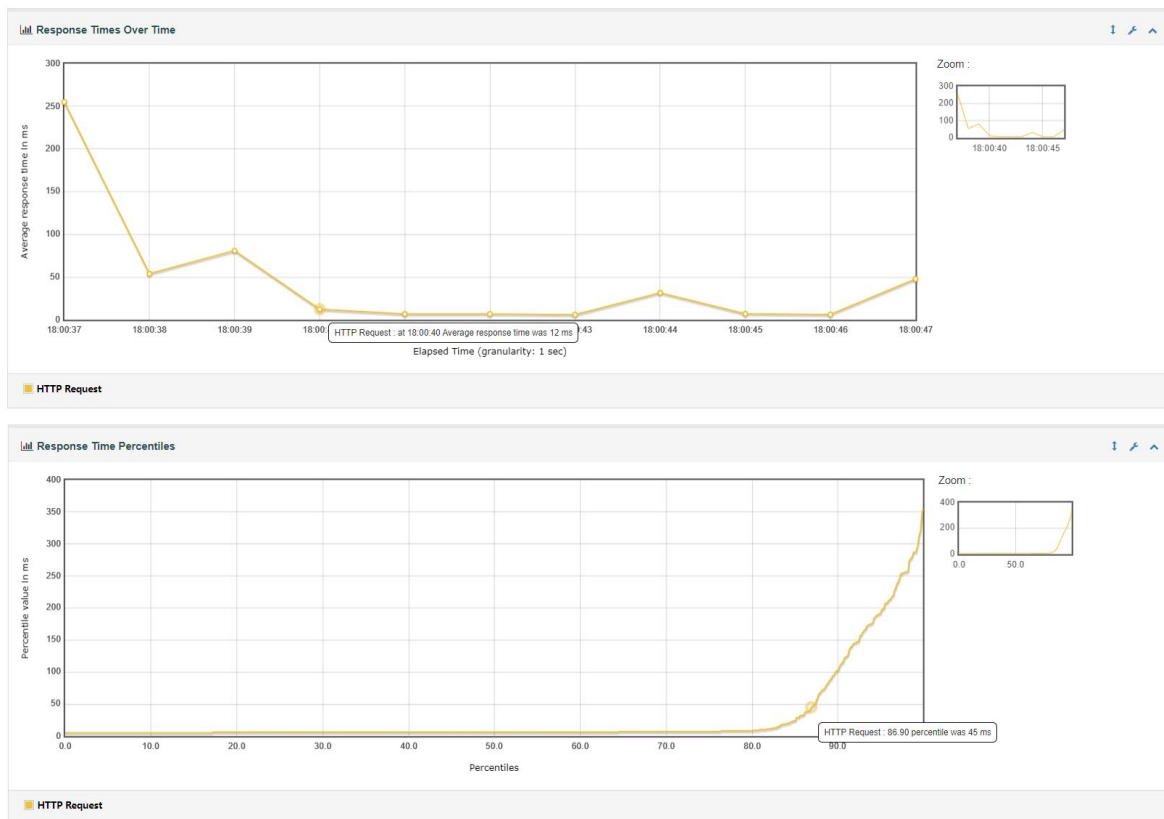


1200-10-1

- Response Times Over Time 继续保持稳定在 10-20ms.
- Response Time Percentiles 轻微下降,89% 的请求在 40ms 内,但整体表现仍然良好,表明系统尚未达到瓶颈.

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	HTTP Request

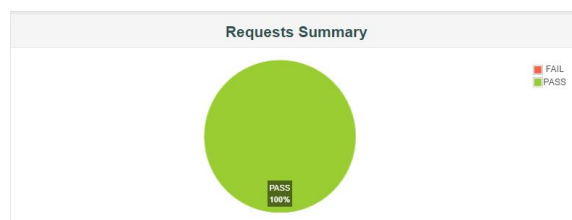


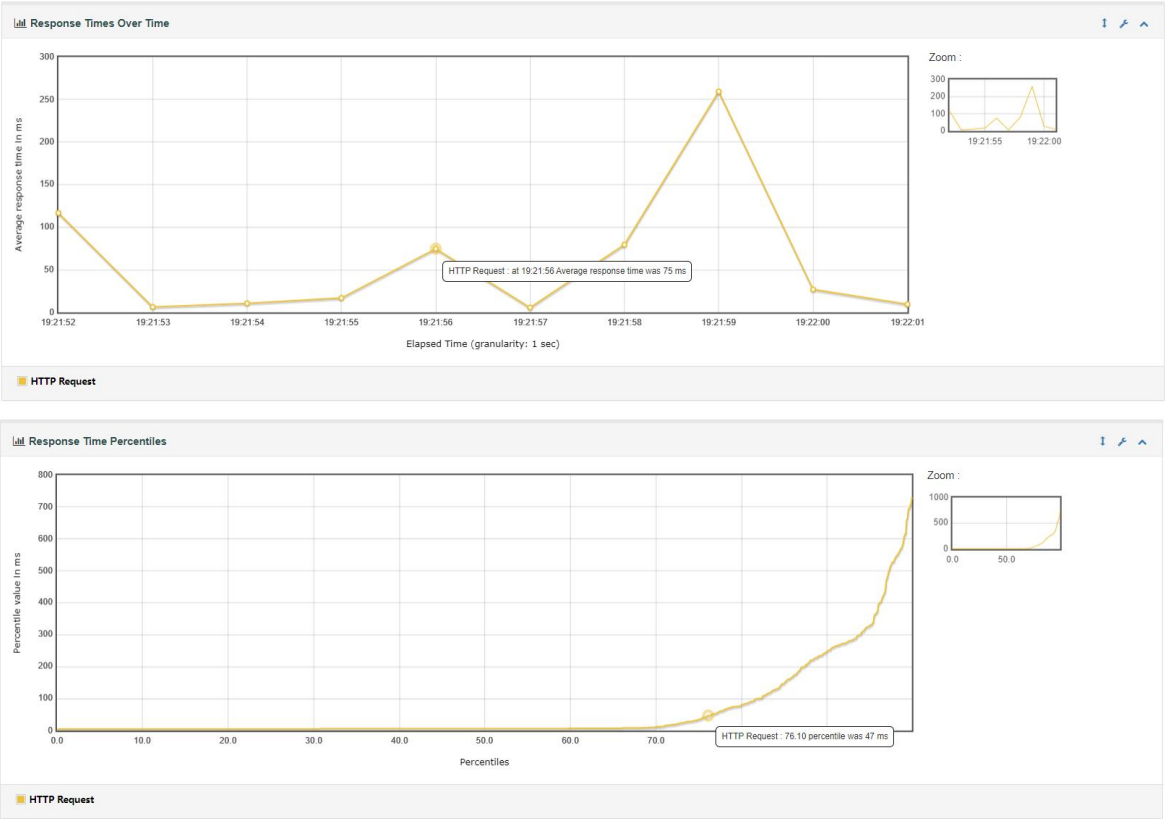


1300-10-1

- **Response Times Over Time** 出现显著波动,延迟增加到 几百甚至上千毫秒,系统开始表现出性能下降的趋势.
- **Response Time Percentiles** 显示仅有 70% 的请求响应时间在 40ms 内,系统已经遇到瓶颈,响应时间大幅增加.

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.986	500 ms	1 sec 500 ms	Total
0.986	500 ms	1 sec 500 ms	HTTP Request





因此,可以推断出系统在 1200-10-1 写请求的并发负载下,是其能有质量地处理请求的最大临界点.在此并发负载下,系统仍然能够维持良好的响应时间和稳定性,超过此负载,系统的响应时间和性能将显著下降.

3 总 结

本次实验对基于 MySQL 的 Spring 应用中的读写效率进行了对比测试,结果表明读操作的效率显著优于写操作.特别是在高并发情况下,写操作由于数据持久化、锁定机制和事务处理等原因,响应时间显著增加.相比之下,读操作能够更好地利用缓存和索引优化,在相同的负载下表现出更好的响应时间和稳定性.实验结果为系统的优化提供了参考,建议在高并发场景下对写操作进行优化,以提高整体性能.

References:

[1] ERINLE B. Performance testing with JMeter [M]. Packt Publishing Ltd, 2015.

[2] GRUNWALD D, NEVES R. Whole-program optimization for time and space efficient threads [J]. ACM SIGOPS Operating Systems Review, 1996, 30(5): 50-9.